

Article

GradFreeBits: Gradient-Free Bit Allocation for Mixed-Precision Neural Networks

Benjamin Jacob Bodner * , Gil Ben-Shalom and Eran Treister * 

Department of Computer Science, Ben-Gurion University, Beer Sheva 8410501, Israel

* Correspondence: benjybo7@gmail.com (B.J.B.); erant@cs.bgu.ac.il (E.T.)

Abstract: Quantized neural networks (QNNs) are among the main approaches for deploying deep neural networks on low-resource edge devices. Training QNNs using different levels of precision throughout the network (mixed-precision quantization) typically achieves superior trade-offs between performance and computational load. However, optimizing the different precision levels of QNNs can be complicated, as the values of the bit allocations are discrete and difficult to differentiate for. Moreover, adequately accounting for the dependencies between the bit allocation of different layers is not straightforward. To meet these challenges, in this work, we propose GradFreeBits: a novel joint optimization scheme for training mixed-precision QNNs, which alternates between gradient-based optimization for the weights and gradient-free optimization for the bit allocation. Our method achieves a better or on par performance with the current state-of-the-art low-precision classification networks on CIFAR10/100 and ImageNet, semantic segmentation networks on Cityscapes, and several graph neural networks benchmarks. Furthermore, our approach can be extended to a variety of other applications involving neural networks used in conjunction with parameters that are difficult to optimize for.

Keywords: neural network compression; quantization; gradient-free optimization; mixed-precision quantization



Citation: Bodner, B.J.; Ben-Shalom, G.; Treister, E. GradFreeBits: Gradient-Free Bit Allocation for Mixed-Precision Neural Networks. *Sensors* **2022**, *22*, 9772. <https://doi.org/10.3390/s22249772>

Academic Editors: Claudio Gennaro and Claudio Vairo

Received: 3 November 2022

Accepted: 9 December 2022

Published: 13 December 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Deep neural networks have been shown to be highly effective in solving many real-world problems. However, deep neural networks often require a large amount of computational resources for both training and inference purposes [1–3]. This limits the adoption and spread of this technology to scenarios with low computational resources.

To mitigate this computational burden, recent efforts have focused on developing specialized hardware to support the computational demands [4] as well as the model compression methods in order to reduce them [5]. These include various techniques such as pruning [6], knowledge distillation [7,8], a neural architecture search (NAS) [9], and as in this paper, quantization [10], which can naturally be combined with other approaches [11].

Quantization methods enable the computations performed by neural networks to be carried out with fixed-point operations rather than floating-point arithmetic [10,12,13]. This improves their computational efficiency and reduces their memory requirements. However, as with other compression methods, this typically comes at the cost of a reduced performance [5]. Recent efforts in the field have focused on improving the trade-offs between model compression and performance by proposing a plethora of quantization schemes tailored for different scenarios.

Quantization schemes can be divided into post-training and quantization-aware training schemes. Post-training schemes decouple the model training and quantization of its weights and/or activations and are most suitable when training data are not available when compressing the network [12,14–16]. Quantization-aware training schemes perform both optimization tasks together and do require training data, which tends to provide a better performance [17–19].